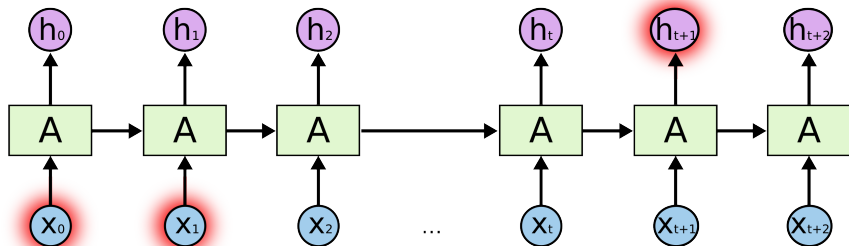


Deep Learning

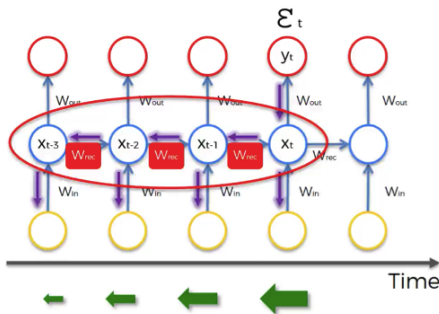
Vazgen Mikayelyan



Problem of Long Term Dependencies



The Vanishing Gradient Problem



$$\frac{\partial \mathcal{E}}{\partial \theta} = \sum_{1 \leq t \leq T} \frac{\partial \mathcal{E}_t}{\partial \theta} \quad (3)$$

$$\frac{\partial \mathcal{E}_t}{\partial \theta} = \sum_{1 \leq k \leq t} \left(\frac{\partial \mathcal{E}_t}{\partial \mathbf{x}_t} \frac{\partial \mathbf{x}_t}{\partial \mathbf{x}_k} \frac{\partial^+ \mathbf{x}_k}{\partial \theta} \right) \quad (4)$$

$$\frac{\partial \mathbf{x}_t}{\partial \mathbf{x}_k} = \prod_{t \geq i > k} \frac{\partial \mathbf{x}_i}{\partial \mathbf{x}_{i-1}} = \prod_{t \geq i > k} \mathbf{W}_{rec}^T \text{diag}(\sigma'(\mathbf{x}_{i-1})) \quad (5)$$

$W_{rec} \sim \text{small} \Rightarrow \text{Vanishing}$
 $W_{rec} \sim \text{large} \Rightarrow \text{Exploding}$

Formula Source: Razvan Pascanu et al. (2013)

- 1 GRU and LSTM
- 2 Bidirectional and Deep RNNs
- 3 Attention Models

Simple RNN



Neural Network
Layer



Pointwise
Operation



Vector
Transfer

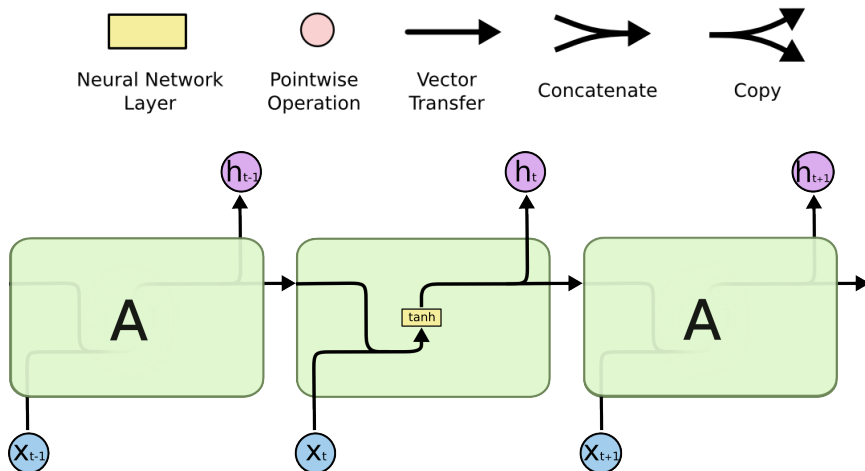


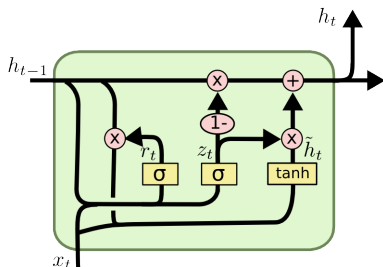
Concatenate



Copy

Simple RNN



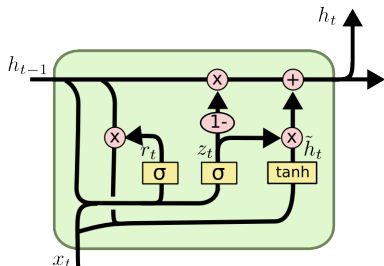


$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$



$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$



Neural Network
Layer



Pointwise
Operation



Vector
Transfer

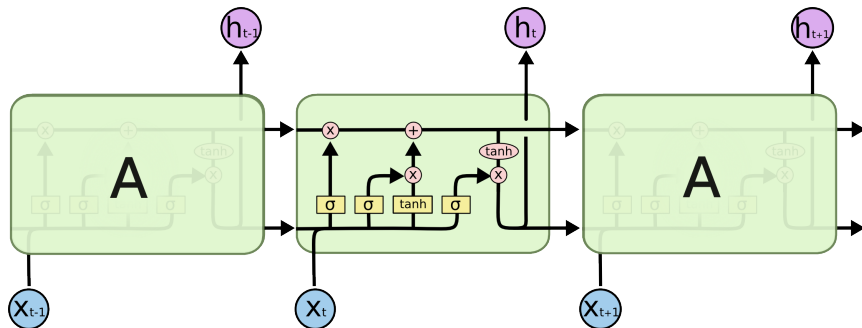


Concatenate

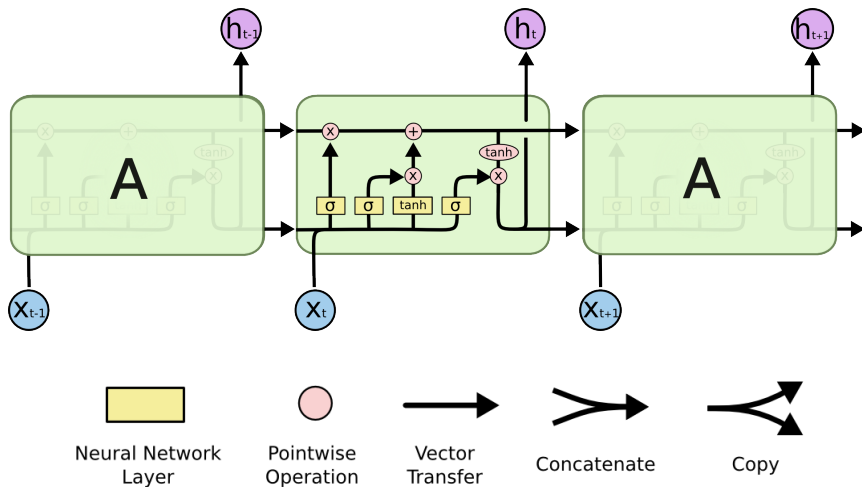


Copy

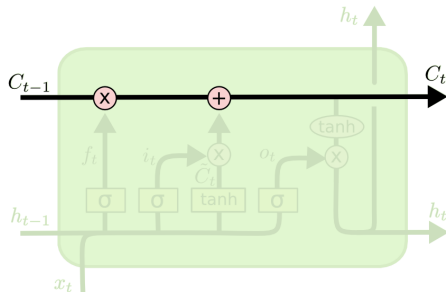
LSTM



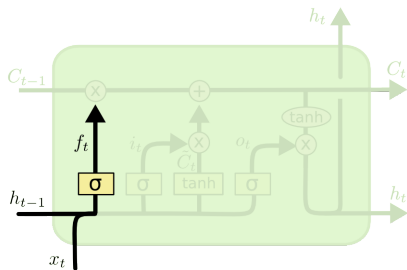
LSTM



Additional state

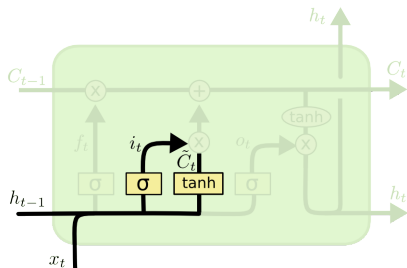


LSTM



$$f_t = \sigma (W_f \cdot [h_{t-1}, x_t] + b_f)$$

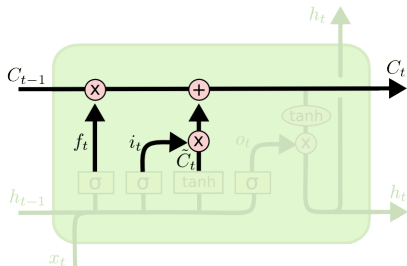
LSTM



$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

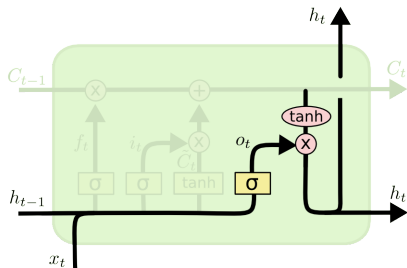
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

LSTM



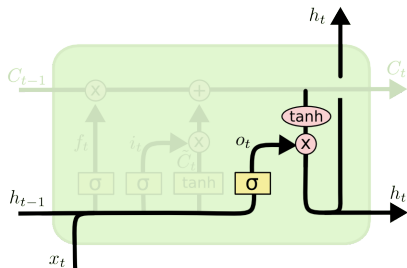
$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

LSTM



$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$



$$o_t = \sigma (W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh (C_t)$$

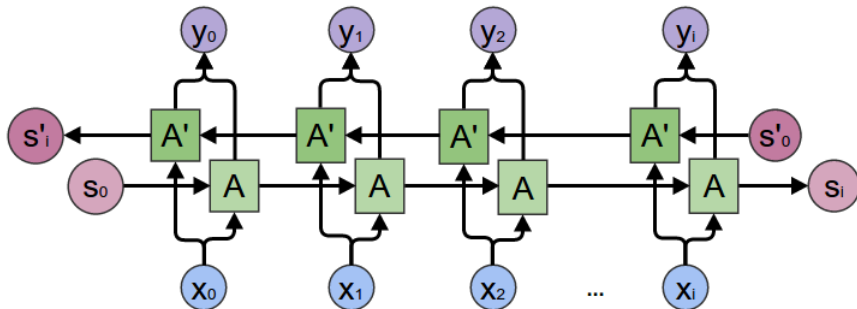
Why to use tanh?

Outline

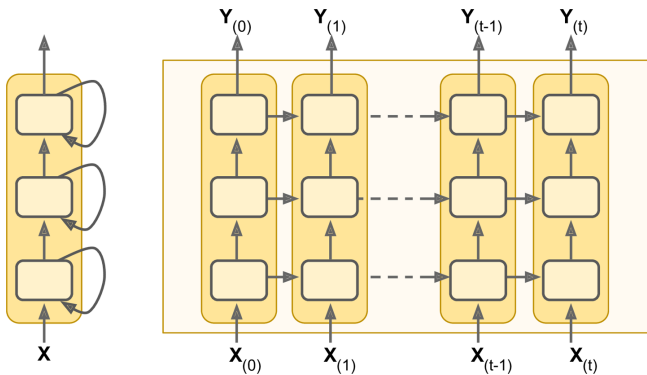
1 GRU and LSTM

2 Bidirectional and Deep RNNs

3 Attention Models



Deep RNNs



Outline

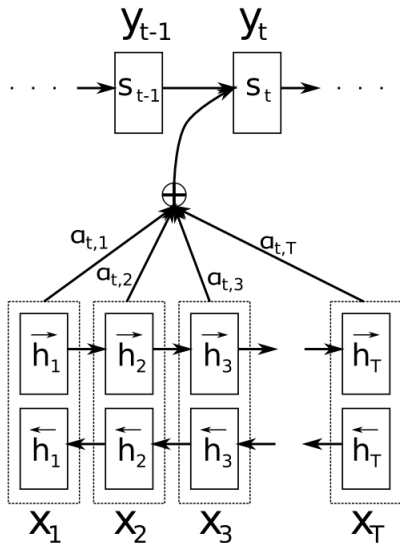
- 1 GRU and LSTM
- 2 Bidirectional and Deep RNNs
- 3 Attention Models**

- Encode each word in the sentence into a vector using RNNs.

- Encode each word in the sentence into a vector using RNNs.
- When decoding, perform a convex combination of these vectors, weighted by “attention weights”.

- Encode each word in the sentence into a vector using RNNs.
- When decoding, perform a convex combination of these vectors, weighted by “attention weights”.
- Use this combination in picking the next word.

Attention Model



Attention Model

Input of decoder is a convex combination of encoded words:

$$c_i = \sum_{j=1}^{T_x} \alpha_{ij} h_j, i = 1, \dots, T_y$$

Attention Model

Input of decoder is a convex combination of encoded words:

$$c_i = \sum_{j=1}^{T_x} \alpha_{ij} h_j, i = 1, \dots, T_y$$

where

$$\alpha_{ij} = \frac{\exp \{e_{ij}\}}{\sum_{j=1}^{T_x} \exp \{e_{ij}\}}$$

Attention Model

Input of decoder is a convex combination of encoded words:

$$c_i = \sum_{j=1}^{T_x} \alpha_{ij} h_j, i = 1, \dots, T_y$$

where

$$\alpha_{ij} = \frac{\exp \{e_{ij}\}}{\sum_{j=1}^{T_x} \exp \{e_{ij}\}}$$

and $e_{i,j}$ are calculated using alignment model

$$e_{ij} = a(s_{i-1}, h_j) = v_a^T \tanh(W_a s_{i-1} + U_a h_j)$$